



Quality Assurance in SW-Development (Practicals)

Assignment 1 “Tutorial” – JaCoCo and Pitest

Summer 2025

What we have learned so far:

"Don't write test cases,
generate them!"

(John Hughes)

What we have learned so far:

"Don't write test cases,
generate them!"

(John Hughes)

But ... Why?

What we have learned so far:

"Don't write test cases,
generate them!"

(John Hughes)

But ... Why?

Good question: YOU will see after this assignment 😊

Your task:

You are given an Actor System and a Message Board in Java
(More on the implementation details will be discussed at the end)

In your groups: Achieve 100% line and branch coverage – but also at least 90% mutation coverage

How can one do this?

Your task:

You are given an Actor System and a Message Board in Java
(More on the implementation details will be discussed at the end)

In your groups: Achieve 100% line and branch coverage – but also at least 90% mutation coverage

How can one do this?

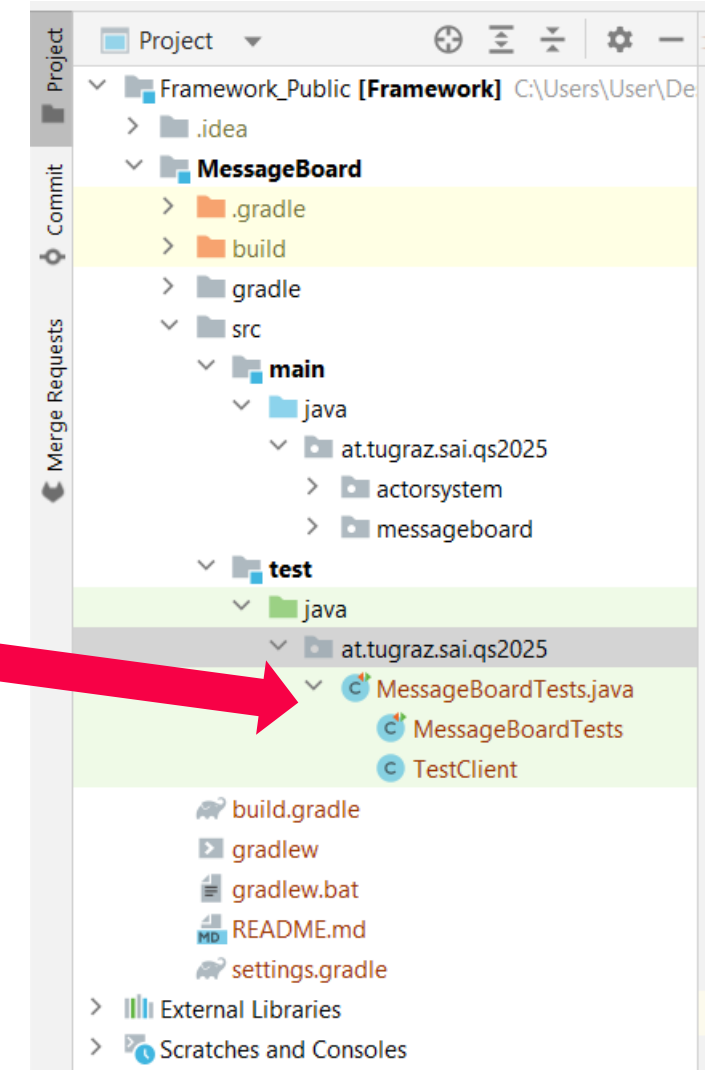
You probably already know ^^

7 Test Cases 😊

Your project structure already is set up for you

Implement all tests in the
MessageBoardTests.java file

Run them with the gradle interface
(Built-in in IntelliJ)



Test Cases Requirements 😊

All test cases need to succeed

All test cases need to have meaningful (!) assertions

There is no minimum number of tests but you will need the specified line and branch coverage (100%) and at least a mutation score of 90%

Test Cases Requirements 😊

All test cases need to succeed

All test cases need to have meaningful (!) assertions

There is no minimum number of tests but you will need the specified line and branch coverage (100%) and at least a mutation score of 90%

How can one check this coverage?

Gradle

After downloading and extracting the .zip → right-click on it and open the folder as an IntelliJ IDEA project

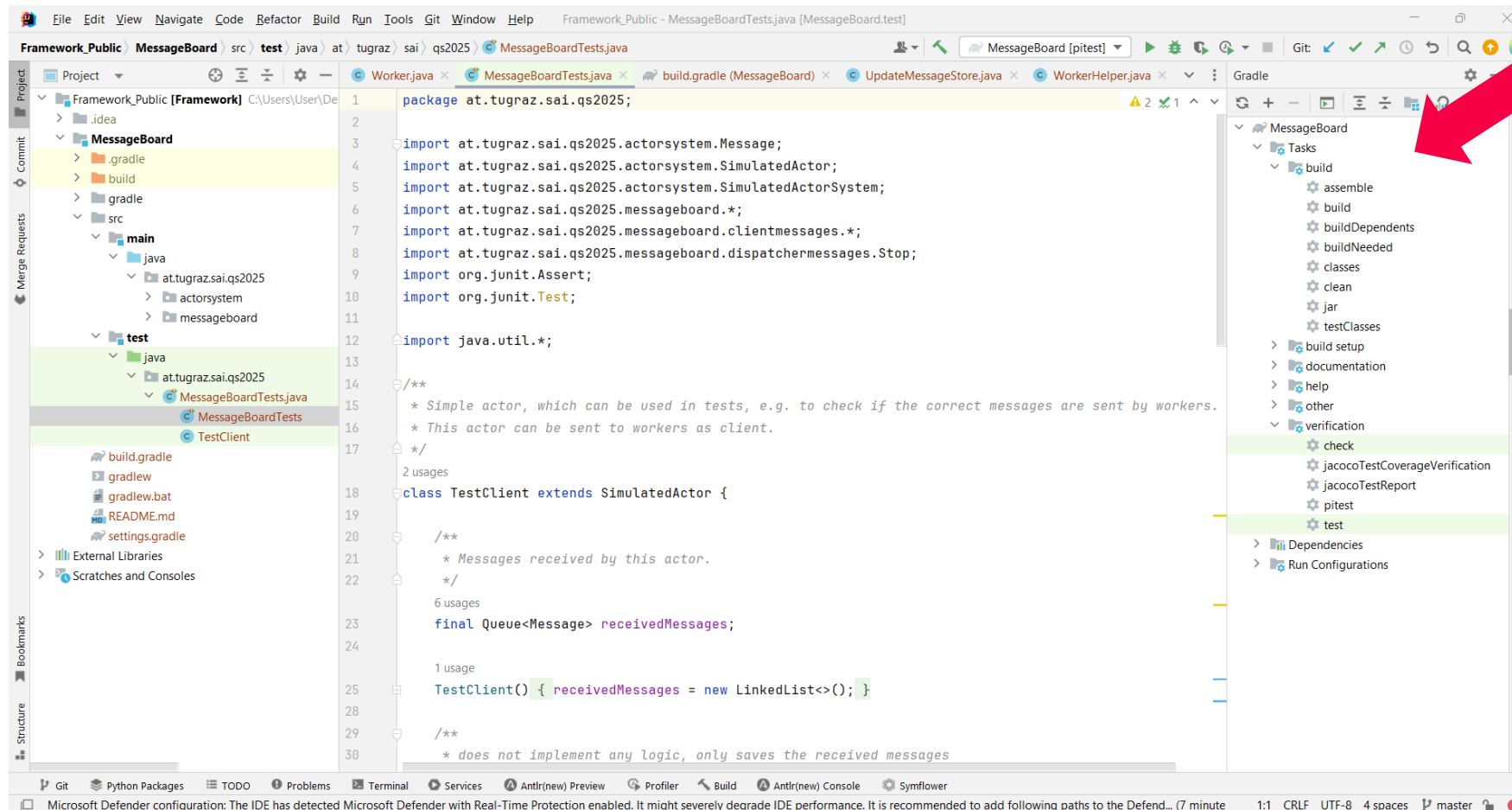
Be sure that you have JDK 17 and IntelliJ recognizes it!

In IntelliJ gradle will likely pop up and ask you if you want to configure it for this project.

After this step, there should be a gradle interface in your IDE

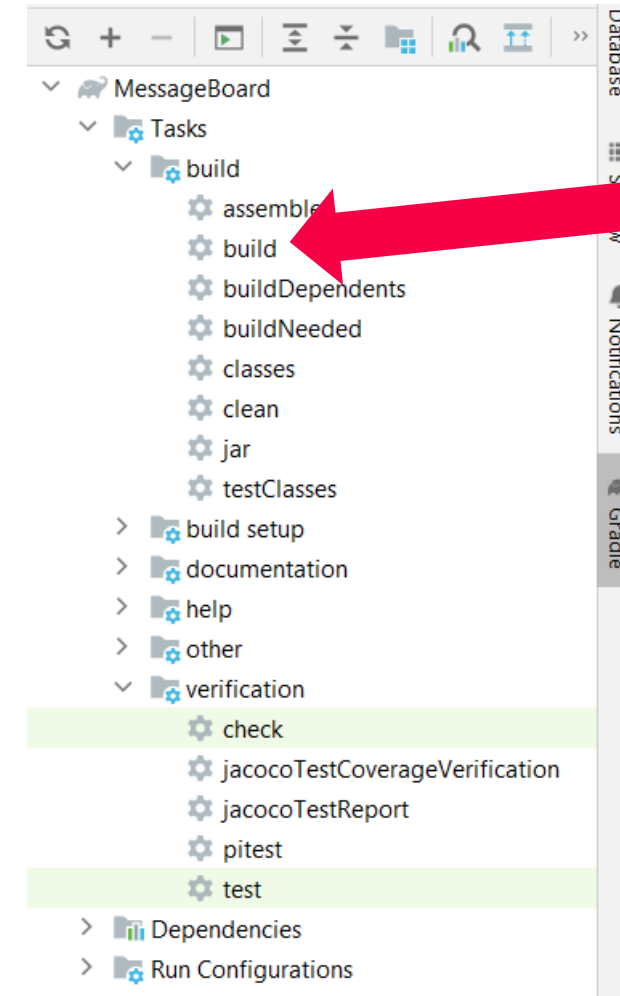
Gradle

The important things you will see:



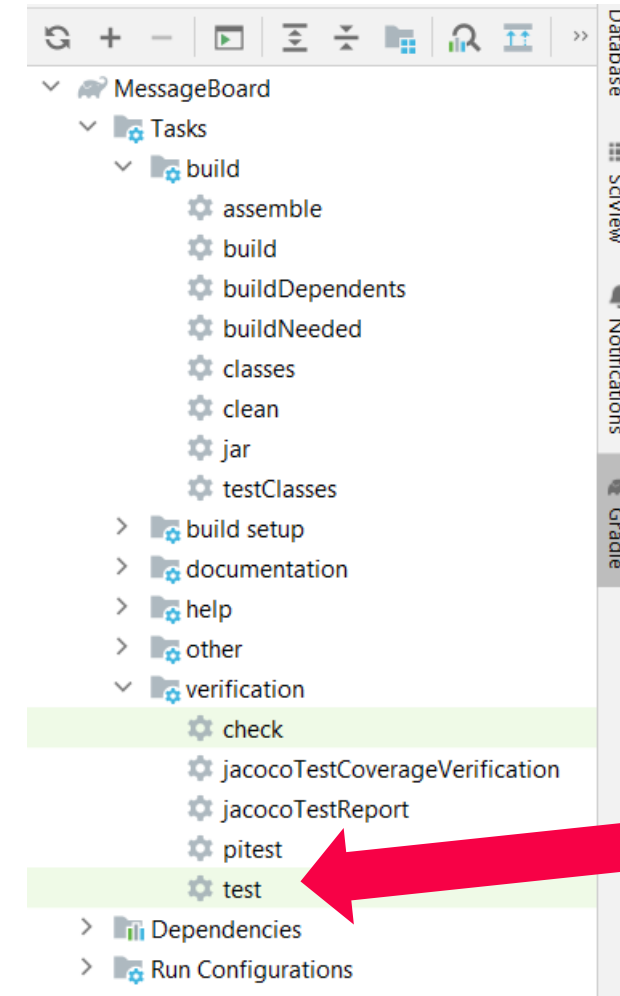
Gradle

Step 1:
Build project



Gradle

Step 2: Run Tests

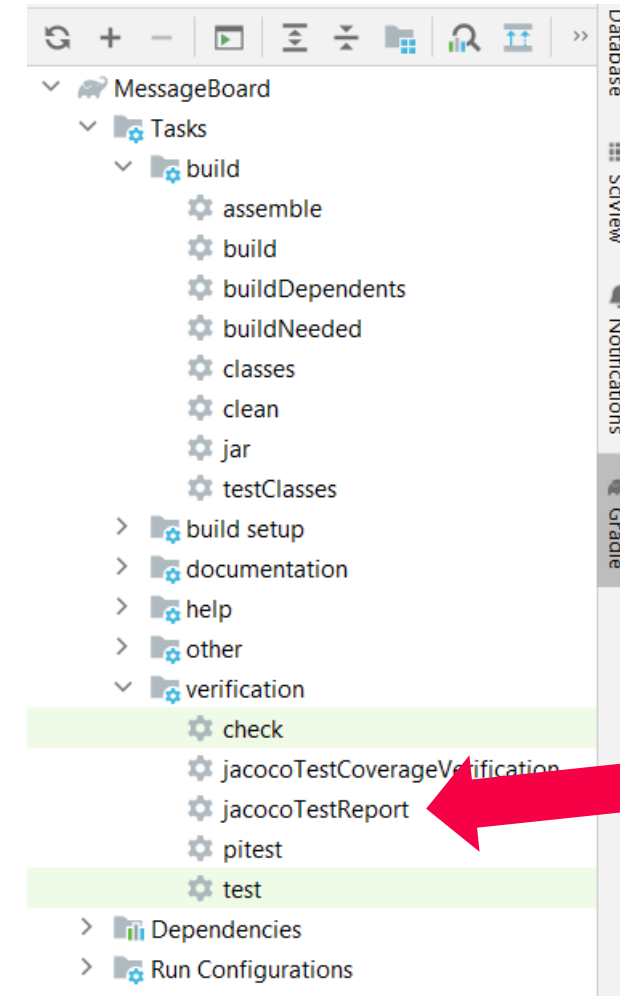


Gradle

Step 3:

Run JaCoCo

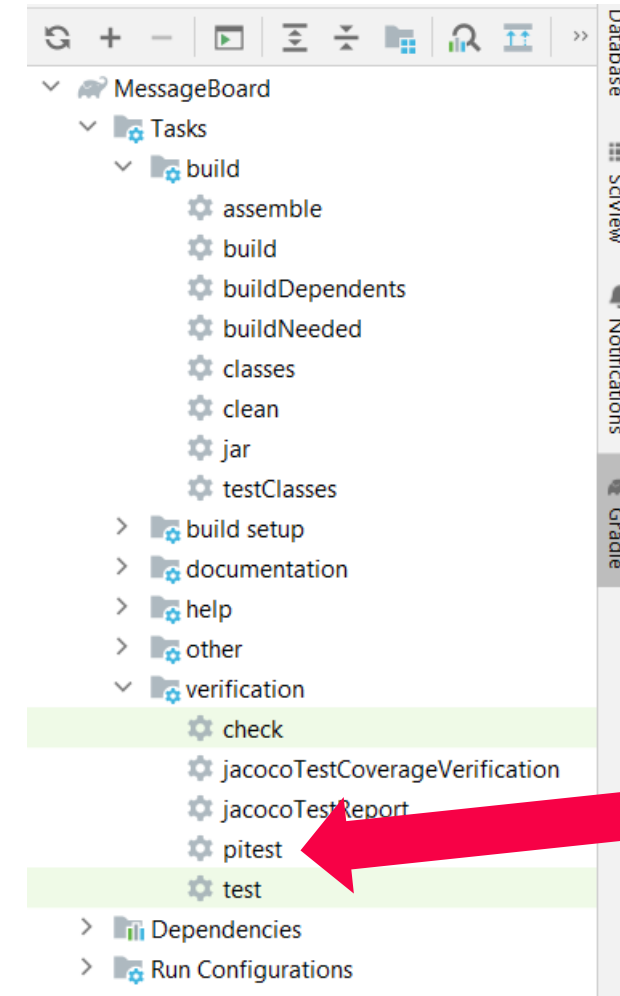
(Code and Branch Coverage)



Gradle

Step 4:

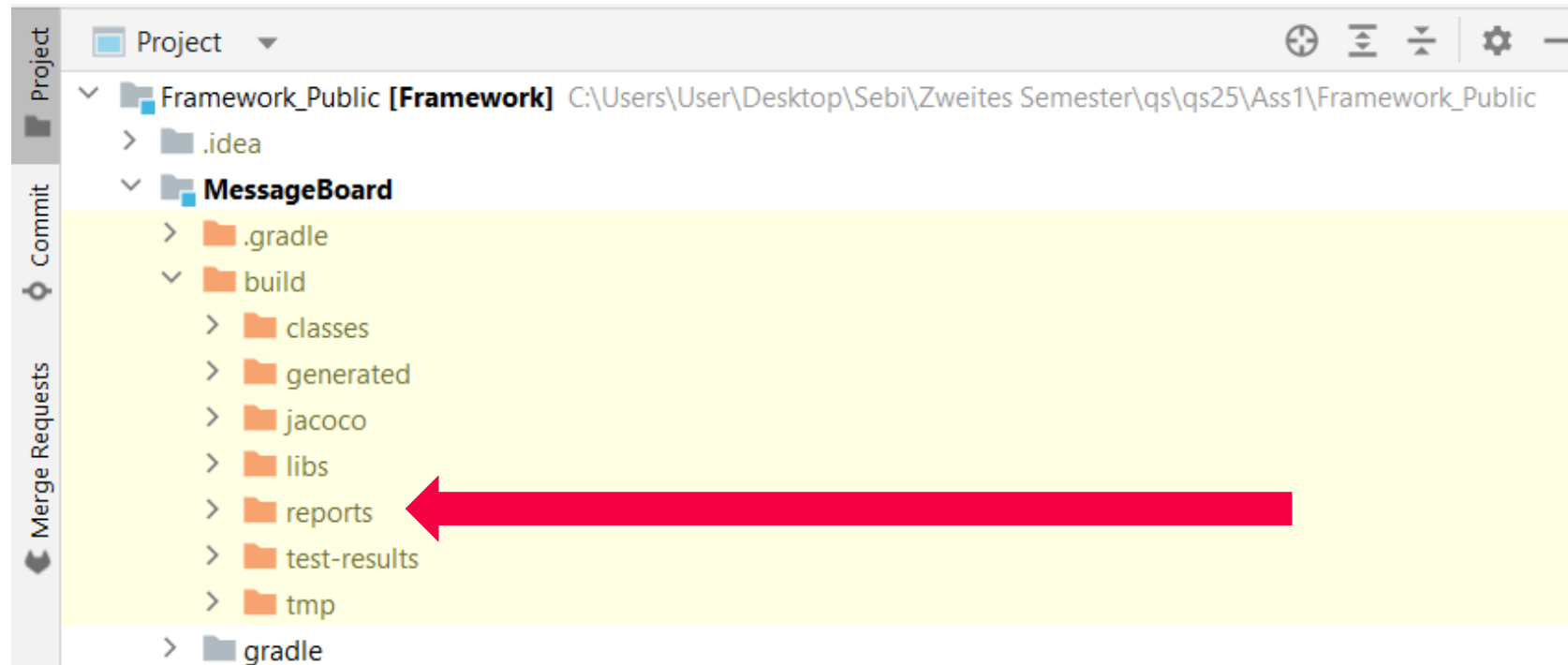
Run Pitest (Mutation Coverage)



JaCoCo – Finding the Report

The report can be found in:

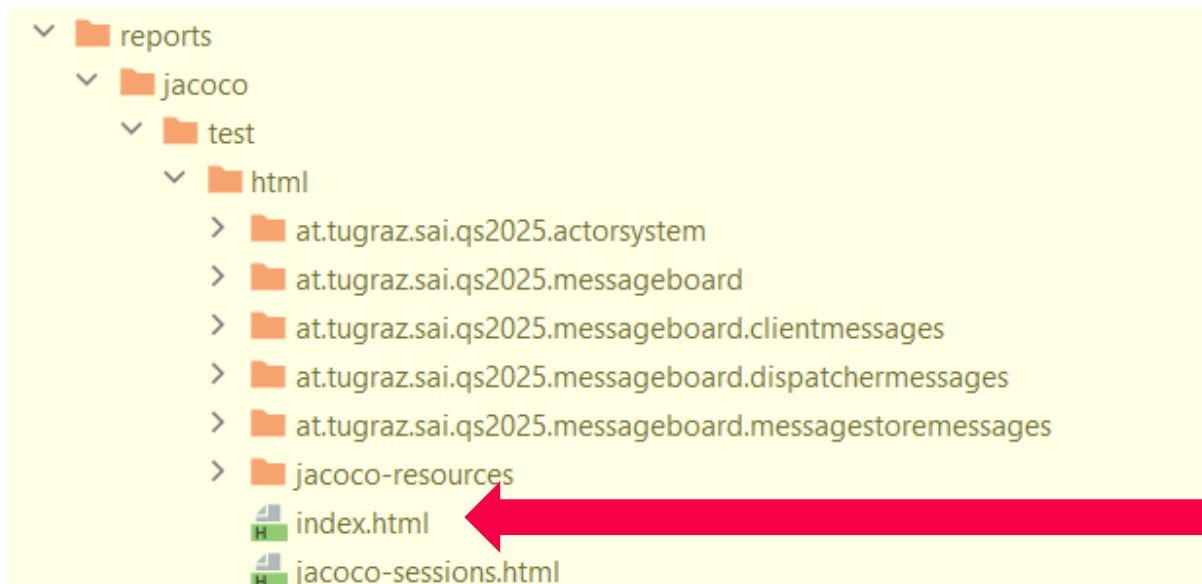
<Path_to_your_project>/qs25_assignment1/MessageBoard/build/reports/jacoco/test/html/index.html



JaCoCo – Finding the Report

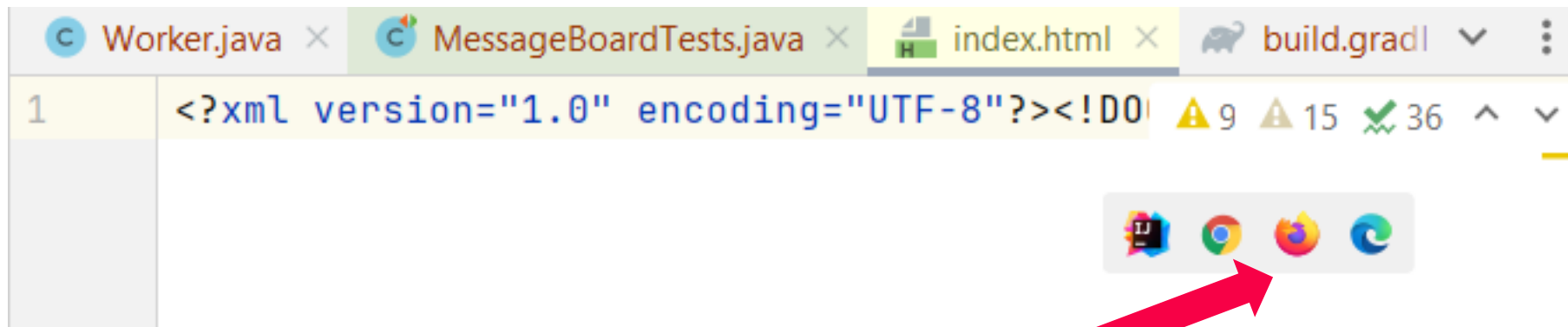
The report can be found in:

<Path_to_your_project>/qs25_assignment1/MessageBoard/build/reports/jacoco/test/html/index.html



JaCoCo – Finding the Report

You can then open the HTML file and see the coverage report.
It is interactive, you can play around and see what you are missing



JaCoCo – Finding the Report

You can then open the HTML file and see the coverage report.
It is interactive, you can play around and see what you are missing

 MessageBoard

MessageBoard

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
at.tugraz.sai.qs2025.messageboard		99 %		99 %	2	157	2	421	0	52	0	8
at.tugraz.sai.qs2025.messageboard.clientmessages		100 %		n/a	0	42	0	104	0	42	0	23
at.tugraz.sai.qs2025.actorsystem		100 %		100 %	0	35	0	87	0	25	0	5
at.tugraz.sai.qs2025.messageboard.messagestoremessages		100 %		n/a	0	12	0	52	0	12	0	11
at.tugraz.sai.qs2025.messageboard.dispatchermessages		100 %		n/a	0	4	0	7	0	4	0	2
Total	4 of 2.879	99 %	2 of 230	99 %	2	250	2	671	0	135	0	49

JaCoCo – Finding the Report

You can then open the HTML file and see the coverage report.
It is interactive, you can play around and see what you are missing

 MessageBoard

MessageBoard

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods	Missed	Classes
at.tugraz.sai.qs2025.messageboard		99 %		99 %	2	157	2	421	0	52	0	8
at.tugraz.sai.qs2025.messageboard.clientmessages		100 %		n/a	0	42	0	104	0	42	0	23
at.tugraz.sai.qs2025.actorsystem		100 %		100 %	0	35	0	87	0	25	0	5
at.tugraz.sai.qs2025.messageboard.messagestoremessages		100 %		n/a	0	12	0	52	0	12	0	11
at.tugraz.sai.qs2025.messageboard.displayermessages		100 %		n/a	0	4	0	7	0	4	0	2
Total	4 of 2.879	99 %	2 of 230	99 %	2	250	2	671	0	135	0	49

JaCoCo – Finding the Report

We can see that we missed some lines and branches ...

MessageStore

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
editMessage(long, String, String)		96 %		90 %	1	6	1	12	0	1
addReaction(String, long, Reaction.Emoji)		96 %		83 %	1	4	1	11	0	1
receive(Message)		100 %		100 %	0	27	0	71	0	1
deleteLikeOrDislike(String, long, RemoveLikeOrDislike.Type)		100 %		100 %	0	6	0	15	0	1
update(UserMessage)		100 %		100 %	0	6	0	12	0	1
addLike(String, long)		100 %		100 %	0	4	0	10	0	1
addDislike(String, long)		100 %		100 %	0	4	0	10	0	1
findByAuthorOrText(String)		100 %		100 %	0	4	0	7	0	1
deleteMessage(String, long)		100 %		100 %	0	3	0	7	0	1
findByAuthor(String)		100 %		100 %	0	3	0	6	0	1
addReport(String, String)		100 %		100 %	0	2	0	5	0	1
MessageStore()		100 %		n/a	0	1	0	6	0	1
isBanned(String)		100 %		100 %	0	3	0	2	0	1
Total	4 of 925	99 %	2 of 120	98 %	2	73	2	174	0	13

JaCoCo – Finding the Report

We can see that we missed some lines and branches ...

MessageStore

Element	Missed Instructions	Cov.	Missed Branches	Cov.	Missed	Cxty	Missed	Lines	Missed	Methods
editMessage(long, String, String)		96 %		90 %	1	6	1	12	0	1
addReaction(String, long, Reaction.Emoji)		96 %		83 %	1	4	1	11	0	1
receive(Message)		100 %		100 %	0	27	0	71	0	1
deleteLikeOrDislike(String, long, Reaction.LikeOrDislike.Type)		100 %		100 %	0	6	0	15	0	1
update(UserMessage)		100 %		100 %	0	6	0	12	0	1
addLike(String, long)		100 %		100 %	0	4	0	10	0	1
addDislike(String, long)		100 %		100 %	0	4	0	10	0	1
findByAuthorOrText(String)		100 %		100 %	0	4	0	7	0	1
deleteMessage(String, long)		100 %		100 %	0	3	0	7	0	1
findByAuthor(String)		100 %		100 %	0	3	0	6	0	1
addReport(String, String)		100 %		100 %	0	2	0	5	0	1
MessageStore()		100 %		n/a	0	1	0	6	0	1
isBanned(String)		100 %		100 %	0	3	0	2	0	1
Total	4 of 925	99 %	2 of 120	98 %	2	73	2	174	0	13

JaCoCo – Finding the Report

Green → All (branches) covered

Yellow → Partial coverages (for branches)

Red → Never reached

```

367.     private boolean editMessage(long messageId, String clientName, String newMessage) {
368.     ◆ if (!messages.containsKey(messageId)) {
369.         return false;
370.     }
371.     ◆ for (UserMessage m : messages.values()) {
372.     ◆ if (m.getAuthor().equals(clientName) && m.getMessage().equals(newMessage)) {
373.         return false;
374.     }
375.     }
376.     UserMessage message = messages.get(messageId);
377.     ◆ if (!clientName.equals(message.getAuthor())) {
378.         return false;
379.     }
380.     message.setMessage(newMessage);
381.     messages.put(message.getMessageId(), message);
382.     return true;
383. }

```

JaCoCo – Finding the Report

Green → All (branches) covered

Yellow → Partial coverages (for branches)

Red → Never reached

```

367.     private boolean editMessage(long messageId, String clientName, String newMessage) {
368.         ◆ if (!messages.containsKey(messageId)) {
369.             return false;
370.         }
371.         ◆ for (UserMessage m : messages.values()) {
372.             ◆ if (m.getAuthor().equals(clientName) && m.getMessage().equals(newMessage)) {
373.                 return false;
374.             }
375.         }
376.         UserMessage message = messages.get(messageId);
377.         ◆ if (!clientName.equals(message.getAuthor())) {
378.             return false;
379.         }
380.         message.setMessage(newMessage);
381.         messages.put(message.getMessageId(), message);
382.         return true;
383.     }

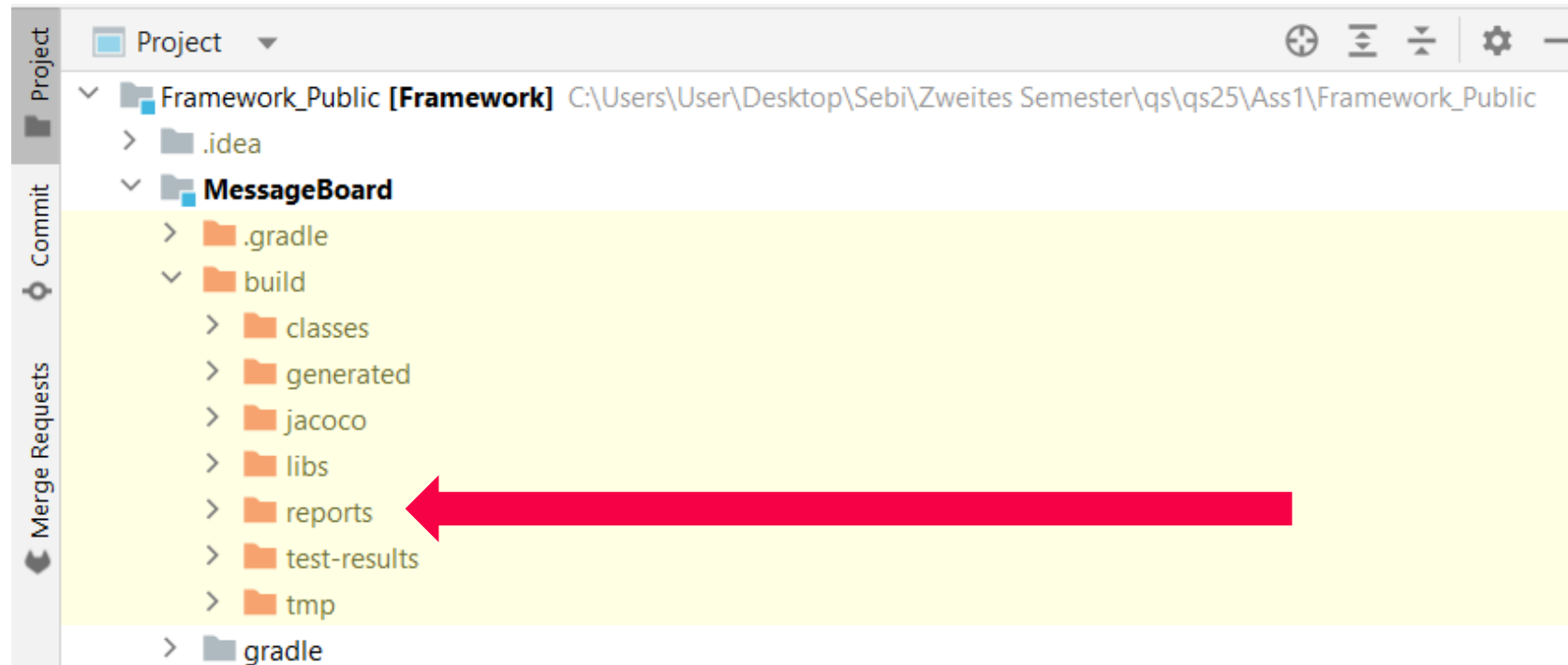
```

Hover over the icon → it will tell you how many branches you missed!

Pitest – Finding the Report

The report can be found in:

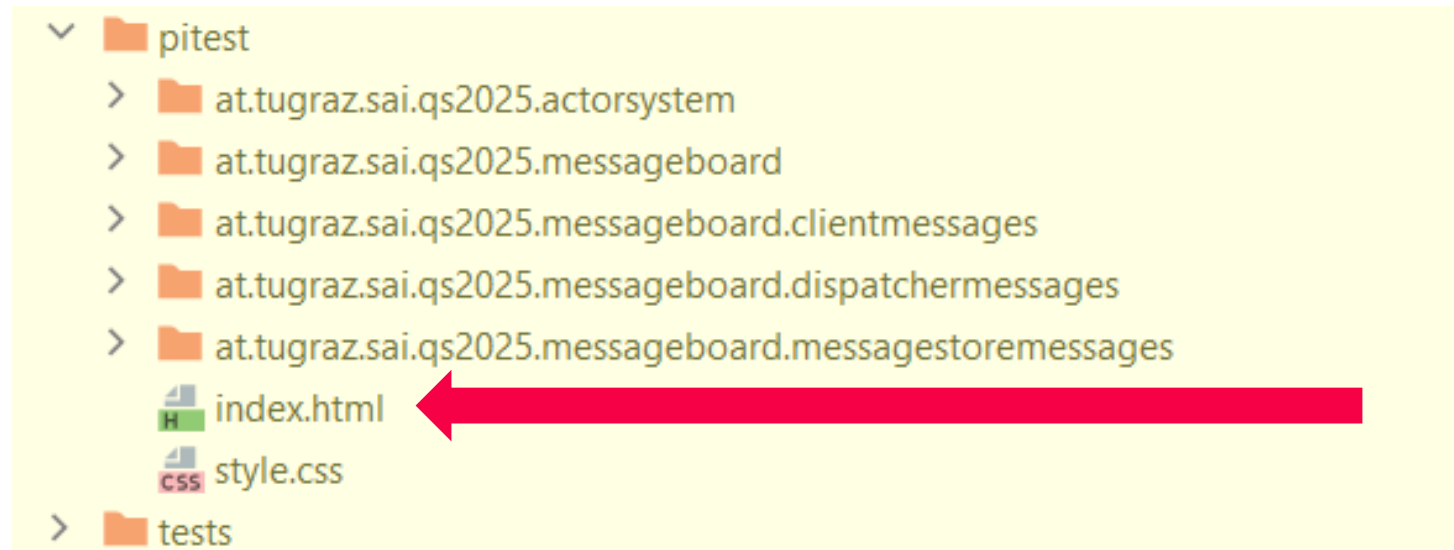
<Path_to_your_project>/qs25_assignment1/MessageBoard/build/reports/pitest/test/html/index.html



Pitest – Finding the Report

The report can be found in:

<Path_to_your_project>/qs25_assignment1/MessageBoard/build/reports/pitest/test/html/index.html



Pitest – Finding the Report

The HTML is pretty similar to the one from JaCoCo (and also has line coverage reports)

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage
30	100% 588/590	93% 356/384

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage
at.tugraz.sai.qs2025.actorsystem	5	100% 87/87	98% 39/40
at.tugraz.sai.qs2025.messageboard	5	100% 414/416	93% 300/324
at.tugraz.sai.qs2025.messageboard.clientmessages	17	100% 78/78	82% 14/17
at.tugraz.sai.qs2025.messageboard.dispatchermessages	2	100% 7/7	100% 2/2
at.tugraz.sai.qs2025.messageboard.messagestoremessages	1	100% 2/2	100% 1/1

Report generated by [PIT](#) 1.5.0

Pitest – Finding the Report

The HTML is pretty similar to the one from JaCoCo (and also has line coverage reports)

Pit Test Coverage Report

Project Summary

Number of Classes	Line Coverage	Mutation Coverage
30	100% 588/590	93% 356/384

Breakdown by Package

Name	Number of Classes	Line Coverage	Mutation Coverage
at.tugraz.sai.qs2025.actorsystem	5	100% 87/87	98% 39/40
at.tugraz.sai.qs2025.messageboard	5	100% 414/416	93% 300/324
at.tugraz.sai.qs2025.messageboard.clientmessages	17	100% 78/78	82% 14/17
at.tugraz.sai.qs2025.messageboard.dispatchermessages	2	100% 7/7	100% 2/2
at.tugraz.sai.qs2025.messageboard.messagestoremessages	1	100% 2/2	100% 1/1

Report generated by [PIT](#) 1.5.0

Pitest – Finding the Report

The HTML also shows the used mutators (highlighted lines and what has been changed)

MessageStoreMessage.java

```

1 package at.tugraz.sai.qs2025.messageboard.messagestoremessages;
2
3 import at.tugraz.sai.qs2025.actorsystem.Message;
4 import at.tugraz.sai.qs2025.actorsystem.SimulatedActor;
5
6 /**
7  * Base class for all messages sent to the message store.
8  */
9 public abstract class MessageStoreMessage implements Message {
10     /**
11      * The actor to which the message store sends its replies.
12      */
13     public SimulatedActor storeClient;
14
15     /**
16      * The id of the communication during which this persistence operation
17      * is performed
18      */
19     public long communicationId;
20
21     public int getDuration() {
22 1 return 1; // store is supposed to be fast
23     }
24 }

```

Mutations

22 1. replaced int return with 0 for at/tugraz/sai/qs2025/messageboard/messagestoremessages/MessageStoreMessage::getDuration → KILLED

Structure of the Code Base

Go through it and compare with the assignment sheet PDF

Try to understand it yourself! Ideally every member, as a similar framework is used for Assignment 2!

Get a good overview of the code base before you start implementing test cases!

→ LIVE DEMO? 😊